

Repaso del día 9

16 preguntas sobre limitaciones de blockchain, escalabilidad, blockchain pública vs permissioned y el ecosistema Hyperledger

Las 16 preguntas — Repaso día 9

1. En blockchain pública sacrificamos rendimiento por descentralización. ¿Qué eje del trilema relaja una red empresarial permissioned y por qué tiene sentido?
2. Bitcoin procesa solo 7 TPS y Ethereum 15-20. Visa hace miles por segundo. ¿Qué hace técnicamente que las blockchains públicas no puedan competir en throughput?
3. Solana presume de hasta 2000 TPS. ¿Por qué entonces el resto de blockchains no copian su receta?
4. ¿Cuál es la diferencia conceptual entre escalar en Layer 1 y en Layer 2?
5. Lightning Network promete pagos instantáneos y casi gratis. ¿Por qué entonces no la usamos para todo?
6. Los rollups optimistic y ZK son los Layer 2 más prometedores. ¿De dónde sacan su seguridad?
7. ¿Por qué una empresa con datos comerciales sensibles no puede usar Ethereum mainnet directamente, ni siquiera con rollups?
8. En Ethereum las identidades son pseudónimas (0x...). ¿Por qué eso es un problema para una empresa regulada?
9. La GDPR consagra el "derecho al olvido". Blockchain promete inmutabilidad. ¿Cómo se reconcilia eso?
10. Tienes una empresa logística con varios almacenes que quiere un sistema de inventario. ¿Por qué blockchain es probablemente mala idea aquí?
11. Cuando alguien dice "voy a desplegar Hyperledger", ¿qué le contestarías?
12. Tu jefe te pide "una blockchain privada compatible con Ethereum". ¿Qué proyecto Hyperledger eliges y por qué NO Fabric?
13. ¿Para qué se diseñó Hyperledger Indy y por qué no es una blockchain de propósito general?
14. Si un proyecto Hyperledger está marcado como "End of Life" (Sawtooth, Burrow), ¿qué significa para un proyecto nuevo?
15. Bitcoin tiene BTC, Ethereum tiene ETH para pagar el gas. Fabric NO tiene criptomoneda propia. ¿Cómo evita entonces el spam y los abusos?
16. ¿En qué lenguajes puedes escribir un chaincode en Fabric?

Pregunta

En blockchain pública sacrificamos rendimiento por descentralización. ¿Qué eje del trilema relaja una red empresarial permissioned y por qué tiene sentido?

Respuesta razonada

Relaja la descentralización: 10.000 nodos anónimos no aportan valor entre 5 empresas que ya se conocen. A cambio gana rendimiento, privacidad y predictibilidad — exactamente lo que un entorno empresarial necesita. No es "mejor" ni "peor", es un trade-off elegido según el caso de uso.

Por qué Bitcoin tiene solo 7 TPS

Pregunta

Bitcoin procesa solo 7 TPS y Ethereum 15-20. Visa hace miles por segundo. ¿Qué hace técnicamente que las blockchains públicas no puedan competir en throughput?

Respuesta razonada

Cada nodo VALIDA cada transacción y el bloque debe propagarse a miles de nodos por todo el mundo antes de que el siguiente se construya encima. La velocidad la marca el nodo más lento de la red, y eso impide subir TPS sin sacrificar descentralización. Por eso bloques más grandes o más rápidos en Layer 1 son siempre un compromiso, no una solución.

Pregunta

Solana presume de hasta 2000 TPS. ¿Por qué entonces el resto de blockchains no copian su receta?

Respuesta razonada

Porque exige nodos potentísimos (multicore, NVMe, ancho de banda >300 Mbps) que solo se permiten unos pocos jugadores; relaja la cantidad de validadores y los criterios de seguridad (no todos los nodos validan). El precio: menos descentralización y ha sufrido varias caídas por bugs en el consenso y saturación. Demuestra que el trilema sigue ahí.

Layer 1 vs Layer 2

Pregunta

¿Cuál es la diferencia conceptual entre escalar en Layer 1 y en Layer 2?

Respuesta razonada

Layer 1 retoca el protocolo base (bloques más grandes, sharding) — caro y arriesgado porque cambia las reglas para todos. Layer 2 deja la cadena base intacta y mueve las transacciones a un sistema lateral que las agrupa y solo ancla resúmenes on-chain. Es más práctico de iterar y donde está la innovación reciente.

Pregunta

Lightning Network promete pagos instantáneos y casi gratis. ¿Por qué entonces no la usamos para todo?

Respuesta razonada

Porque exige bloquear fondos por adelantado entre dos partes y solo sirve dentro de ese canal. Es ideal para flujos repetidos entre pocos actores (pagos recurrentes entre el mismo cliente y comercio), pero no para enviar dinero a alguien aleatorio una sola vez. La UX es muy distinta a una transferencia bancaria.

Pregunta

Los rollups optimistic y ZK son los Layer 2 más prometedores. ¿De dónde sacan su seguridad?

Respuesta razonada

De la cadena base (Ethereum), donde anclan periódicamente sus pruebas. No dependen de confiar en ningún operador: en Optimistic cualquiera puede impugnar un fraude en una ventana de 7 días; en ZK la matemática garantiza la validez al instante. Por eso son más fiables que las sidechains, que tienen su propio consenso independiente.

Pregunta

¿Por qué una empresa con datos comerciales sensibles no puede usar Ethereum mainnet directamente, ni siquiera con rollups?

Respuesta razonada

Porque en una red pública TODA transacción es visible para cualquiera. Tu competencia ve tus contrapartes, volúmenes y patrones operativos. Una blockchain permissioned permite restringir quién ve qué desde el diseño (canales privados en Fabric, PDCs) — algo que en pública solo se logra con criptografía pesada (zk-proofs) que aún no es viable para todo caso de uso.

Pregunta

En Ethereum las identidades son pseudónimas (0x...). ¿Por qué eso es un problema para una empresa regulada?

Respuesta razonada

Porque la regulación financiera (KYC/AML) exige saber quién está al otro lado. Un pseudónimo no encaja: si pasa algo, no sabes a quién perseguir. Las empresas necesitan identidades verificadas (certificados X.509 o equivalente) ligadas a una persona o entidad real, no a un hash de clave anónimo.

Pregunta

La GDPR consagra el "derecho al olvido". Blockchain promete inmutabilidad. ¿Cómo se reconcilia eso?

Respuesta razonada

Mal. En una red pública no se puede borrar nada, lo que choca de frente con la GDPR. La solución típica en redes empresariales es NO meter datos personales en la cadena: solo hashes o referencias, dejando los datos reales en bases off-chain donde sí puedes borrarlos. La blockchain prueba que el dato existió y no se ha alterado, pero el dato real vive fuera.

Pregunta

Tienes una empresa logística con varios almacenes que quiere un sistema de inventario. ¿Por qué blockchain es probablemente mala idea aquí?

Respuesta razonada

Porque solo hay UN actor (la propia empresa) — no hay falta de confianza entre partes que blockchain pueda resolver. Una base de datos tradicional es más simple, más barata y más rápida. Blockchain solo aporta valor cuando hay múltiples actores que NO se fían ciegamente entre sí. Aplicar blockchain a un problema interno suele ser tecnología buscando problema.

Qué es Hyperledger

Pregunta

Cuando alguien dice "voy a desplegar Hyperledger", ¿qué le contestarías?

Respuesta razonada

Que se aclare: Hyperledger no es una tecnología, es un PARAGUAS de proyectos open-source de la Linux Foundation (hoy LF Decentralized Trust): Fabric, Besu, Indy, Iroha, Cacti, Caliper, etc. Decir "Hyperledger" sin más es como decir "Apache" sin especificar HTTP, Kafka o Spark — no significa nada concreto.

Pregunta

Tu jefe te pide "una blockchain privada compatible con Ethereum". ¿Qué proyecto Hyperledger eliges y por qué NO Fabric?

Respuesta razonada

Besu. Es un cliente Ethereum real (EVM, Solidity, MetaMask, herramientas Ethereum) adaptado a entornos permissioned. Fabric NO tiene relación con Ethereum: su modelo, su lenguaje (Go/Node/Java para chaincode) y su arquitectura (Execute-Order-Validate, canales) son totalmente distintos. Elegir uno u otro depende de si traes una pila Ethereum existente o partes de cero.

Pregunta

¿Para qué se diseñó Hyperledger Indy y por qué no es una blockchain de propósito general?

Respuesta razonada

Para gestionar identidades digitales descentralizadas (SSI/DIDs). No tiene tokens ni smart contracts arbitrarios: hace una cosa concreta — registrar quién puede emitir credenciales válidas y sus revocaciones. Especializarse es precisamente lo que la hace simple y robusta para ese caso de uso, en lugar de ser una blockchain de propósito general más.

Pregunta

Si un proyecto Hyperledger está marcado como "End of Life" (Sawtooth, Burrow), ¿qué significa para un proyecto nuevo?

Respuesta razonada

Que no debes empezar nada nuevo con él: la comunidad lo abandonó, no recibe parches de seguridad y no hay soporte. El código sigue accesible por contexto histórico, pero la recomendación oficial es migrar: Sawtooth → Fabric o Besu, Burrow → Besu, Ursa → AnonCreds/Indy/Aries, Avalon → Confidential Computing Consortium.

Fabric sin criptomoneda

Pregunta

Bitcoin tiene BTC, Ethereum tiene ETH para pagar el gas. Fabric NO tiene criptomoneda propia. ¿Cómo evita entonces el spam y los abusos?

Respuesta razonada

No los necesita porque NO es una red abierta: solo los participantes con certificado emitido por una CA reconocida pueden enviar transacciones. Quien abusa, queda identificado y se le revoca. El gas existe en Ethereum precisamente porque cualquiera puede entrar — en una red permissioned el problema desaparece por construcción y los costes de operación son fijos y predecibles.

Pregunta

¿En qué lenguajes puedes escribir un chaincode en Fabric?

Respuesta

Go, Node.js / TypeScript o Java

Hyperledger Fabric

La blockchain empresarial: cómo funciona y por qué se ha convertido en el estándar de facto

¿Por qué blockchain empresarial?

Imagínate que tres bancos quieren registrar transacciones de manera conjunta. Necesitan que:

- ✓ Ninguno de los tres pueda alterar el registro a posteriori
- ✓ Todos vean los datos en tiempo real
- ✓ La identidad de cada participante esté verificada (no anónima)
- ✓ Algunos datos sean privados entre subgrupos
- ✓ Rendimiento real, no 15 transacciones por segundo

Ethereum no encaja: es público (cualquiera ve todo, cualquiera entra), lento (~15 TPS), caro (gas), y las identidades son pseudónimas.
Hyperledger Fabric existe para resolver justamente esto.

Aspecto	Ethereum (lo que ya conoces)	Hyperledger Fabric
Acceso	Permissionless: cualquiera entra	Permissioned: solo participantes autorizados
Identidad	Pseudónima (0x...)	Identidad real con certificados X.509
Moneda nativa	ETH, pagas gas en cada operación	No hay. Sin gas, sin volatilidad
Privacidad	Todo público	Canales y datos privados por grupo
Smart contracts	Solidity sobre la EVM	Chaincode en Go, Node.js o Java
Rendimiento	~15-20 TPS	Cientos a miles de TPS

Idea clave: *Fabric NO compite con Ethereum, resuelve un problema distinto. Donde Ethereum dice "que cualquiera entre y todo sea público", Fabric dice "que entren solo los autorizados y compartan lo necesario".*

"

Una plataforma blockchain modular y permissionada para crear redes

"

empresariales entre organizaciones que se conocen pero no se fían entre sí.

Las tres palabras clave

Modular — puedes cambiar piezas (consenso, base de datos, identidad) sin reescribir la aplicación.

Permissionada — no cualquiera puede entrar. Cada participante tiene una identidad emitida por una autoridad certificadora de su organización.

Empresarial — diseñada para casos reales de negocio: trazabilidad, supply chain, finanzas interbancarias, registros médicos, no para criptomonedas.

Las 4 piezas que mueven Fabric (vista de pájaro)

Antes de entrar en detalle, quédate con esta visión simplificada. Todo lo demás son matices sobre esto:



CLIENTE

La aplicación que inicia las transacciones. No ejecuta lógica, solo orquesta.



PEERS

Los nodos que ejecutan el chaincode y guardan el ledger. El motor del sistema.



ORDERER

Ordena las transacciones validadas y las agrupa en bloques. No decide qué es válido.



LEDGER

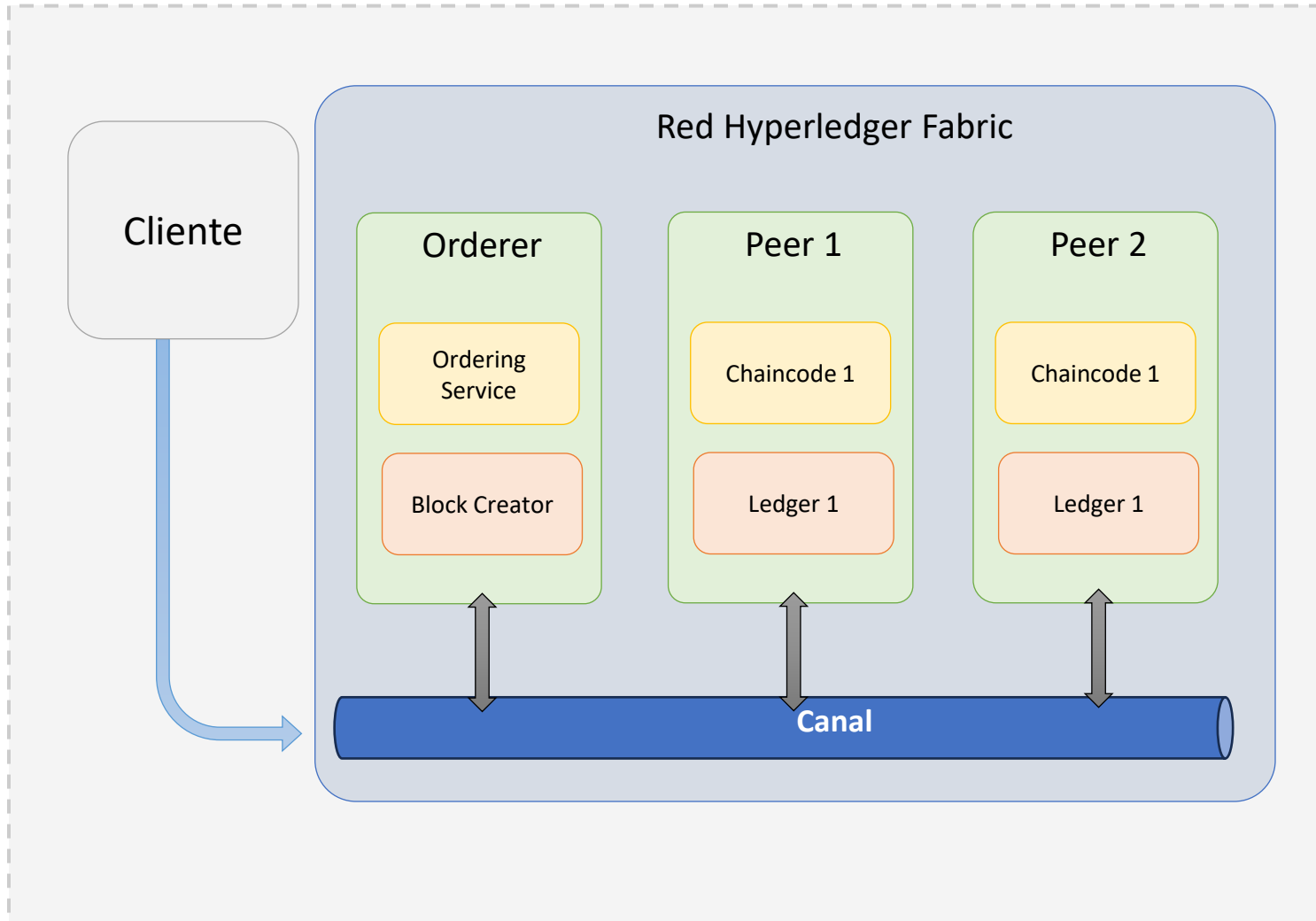
La base de datos distribuida con el histórico (blockchain) y el estado actual (world state).

Todos los componentes de Fabric

Las 4 piezas anteriores son lo central, pero Fabric tiene más componentes que dan estructura, identidad y privacidad a la red:

Componente	Para qué sirve
 Cliente	Aplicación que inicia transacciones
 Peers	Nodos que ejecutan chaincode y mantienen el ledger
 Orderer	Ordena transacciones y crea bloques
 Chaincode	La lógica de negocio (≈ smart contracts)
 Ledger	Base de datos con blockchain + world state
 CA	Emite certificados X.509 a los participantes
 MSP	Define qué identidades son válidas
 Canal	Subred privada con su propio ledger
 PDC	Datos privados visibles solo para algunos peers del canal

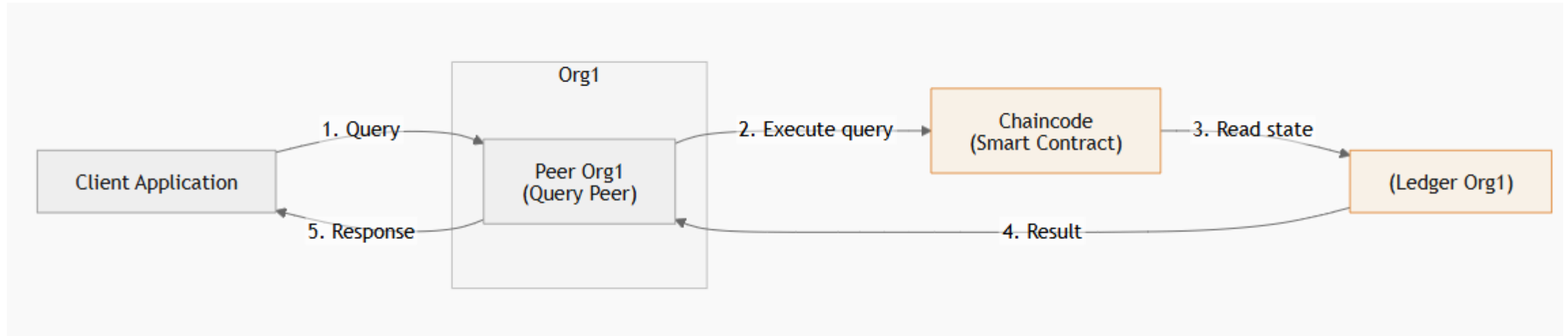
Vista simplificada de una red Fabric



Lo que ves en el diagrama

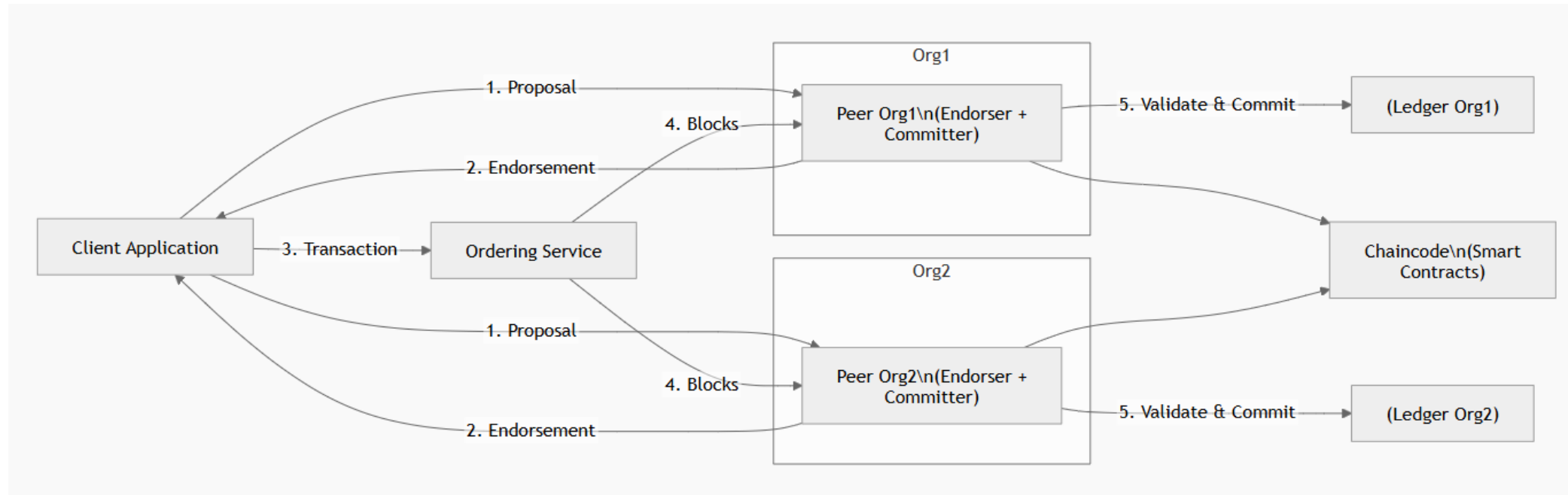
- **Cliente:** aplicación que envía solicitudes
- **Peer:** ejecuta chaincode y mantiene ledger
- **Chaincode:** lógica de negocio compartida
- **Ledger:** estado actual + histórico
- **Canal:** espacio privado de comunicación
- **Orderer:** ordena transacciones, crea bloques
- **Red Fabric:** conjunto de nodos y servicios

Flujo resumido de una aplicación Fabric (lectura)



1. La aplicación cliente envía una query a un peer.
2. Ese peer ejecuta el chaincode correspondiente en el propio peer.
3. El chaincode lee datos del ledger local del propio peer.
4. El chaincode devuelve el resultado.
5. El peer responde al cliente.

Flujo resumido de una aplicación Fabric (escritura)



Qué es

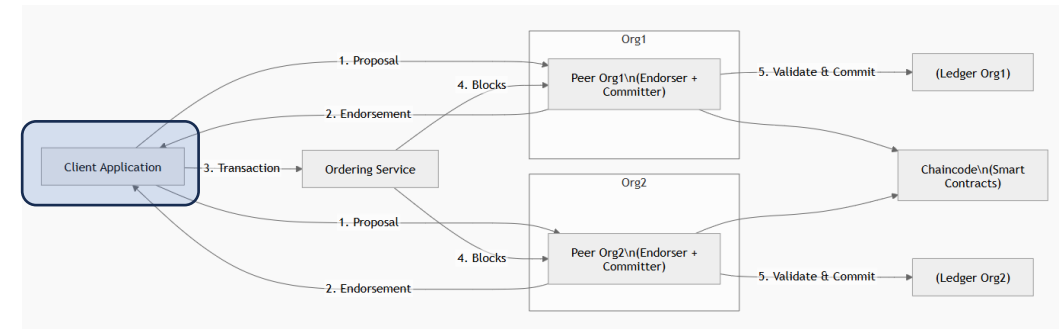
- Aplicación que inicia transacciones
- Usa SDK (Node, Java, Go)

Qué hace

- Construye propuestas de transacción
- Las firma con su identidad (certificado X.509)
- Las envía a los peers (endorsers)
- Recoge las respuestas y las envía al orderer

Importante

- X NO ejecuta chaincode
- X NO accede al ledger directamente



Idea clave: El cliente orquesta, pero no ejecuta ni valida.

Qué son

Nodos que mantienen el ledger y ejecutan la lógica.

Roles que puede asumir un peer

- **Anchor peer** → Peer público conocido por otras organizaciones
- **Leader peer** → Recibe bloques del orderer y los distribuye
- **Endorser** → Recibe propuestas y ejecuta chaincode (simulación)
- **Committer** → Valida bloques y los escribe en el ledger

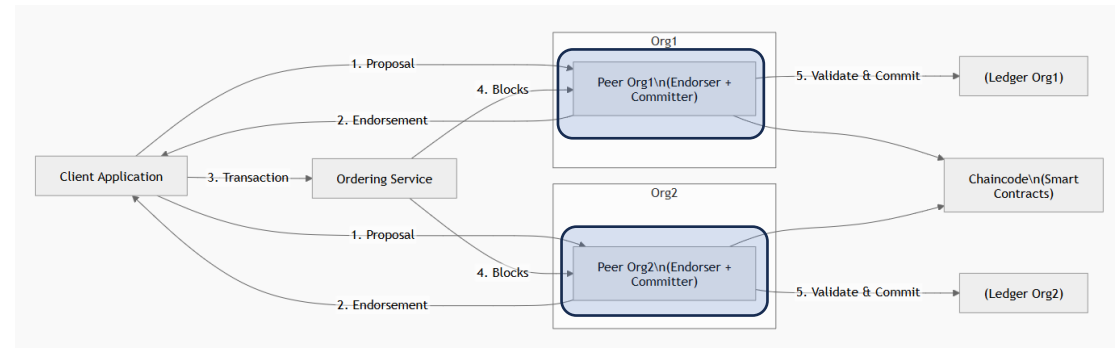
Un mismo peer puede asumir varios roles a la vez.

Detalle clave

Cada peer guarda UN ledger por cada canal en el que participa.

Los endorsers:

1. verifican identidad y firma del cliente,
2. ejecutan el chaincode,
3. comprueban reglas de negocio,
4. leen el ledger actual y general el conjunto de datos leídos para la transacción y el de datos escritos,
5. firman el resultado.



Idea clave: Los peers son el motor del sistema. Ejecutan, validan y guardan.

Qué es

Servicio que ordena las transacciones validadas.

Qué hace

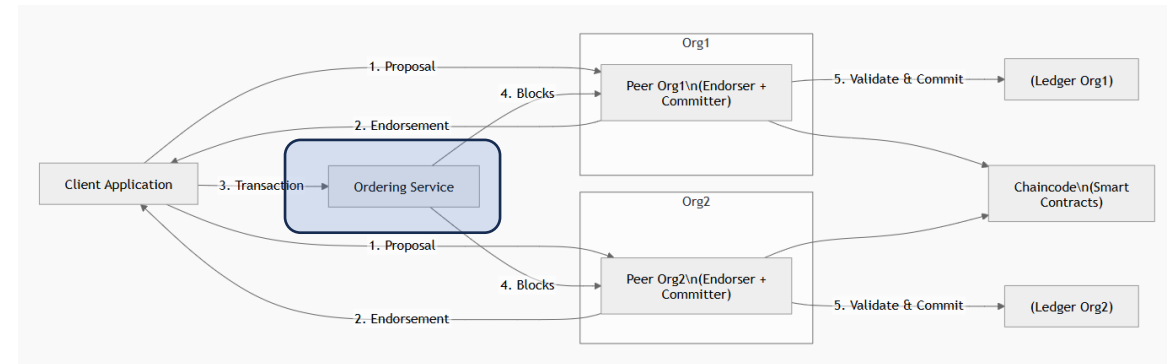
- Recibe transacciones ya endosadas por los peers
- Las ordena (por timestamp y otros criterios)
- Las empaqueta en bloques y los distribuye
- Gestiona la configuración de cada canal

Importante

- X NO ejecuta chaincode
- X NO valida la lógica de negocio

Gobernanza

Operado por varias organizaciones (consenso Raft) para que ninguna controle el orden sola.



Idea clave: El orderer no decide qué es válido. Solo decide el orden.

El Chaincode (Smart Contracts de Fabric)

Qué es

La lógica de negocio. Equivalente a un smart contract en Ethereum.

Dónde vive

En los peers (como contenedor o proceso). Cuando un peer ejecuta una transacción, llama al chaincode.

Cómo se invoca

Cliente → peer → chaincode

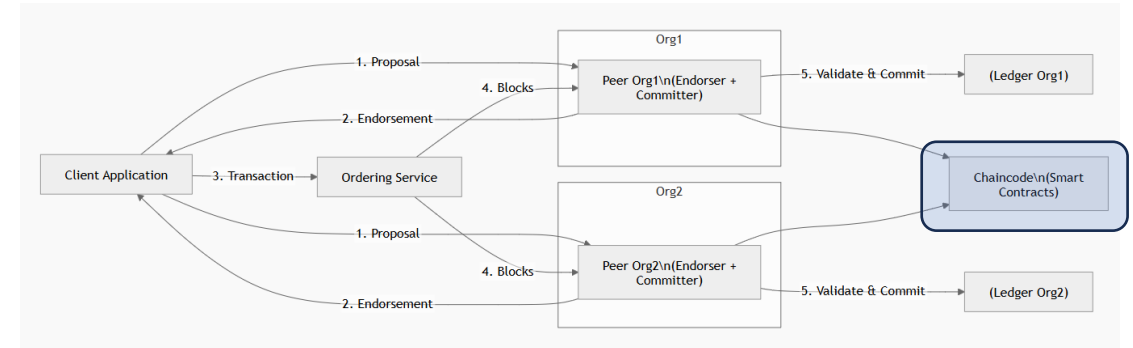
El cliente especifica el nombre de la función y los argumentos.

Importante

X NO tiene endpoints propios (no es un servidor HTTP)

X NO se ejecuta en el cliente

Lenguajes: Go, Node.js / TypeScript, Java.



Idea clave: El chaincode define las reglas, pero el peer las ejecuta.

Esta es probablemente LA diferencia más importante entre Fabric y Ethereum:

Ethereum / blockchain pública

TODOS los nodos ejecutan TODO el código
de TODAS las transacciones.

Ineficiente pero seguro: nadie depende de nadie en particular.

Hyperledger Fabric

Solo los ENDORSERS designados ejecutan el chaincode.
Una política define cuántos y cuáles.

Eficiente y selectivo: si los endorsers requeridos validan, vale para
toda la red.

¿Qué es una endorsement policy?

Cada chaincode tiene asociada UNA endorsement policy que define qué peers (y de qué organizaciones) deben firmar una transacción para que se considere válida.

Ejemplos típicos:

- `AND('Org1.peer', 'Org2.peer')` → ambas orgs deben endosar (típico en transacciones bilaterales)
- `OR('Org1.peer', 'Org2.peer')` → basta con que una endose
- `OutOf(2, 'Org1.peer', 'Org2.peer', 'Org3.peer')` → 2 de 3 (mayoría)

Idea clave: no hace falta que TODOS ejecuten el chaincode. Si los endorsers exigidos por la política firman, la transacción vale para toda la red.

Qué es

La base de datos distribuida que guarda el estado del sistema.

Componentes

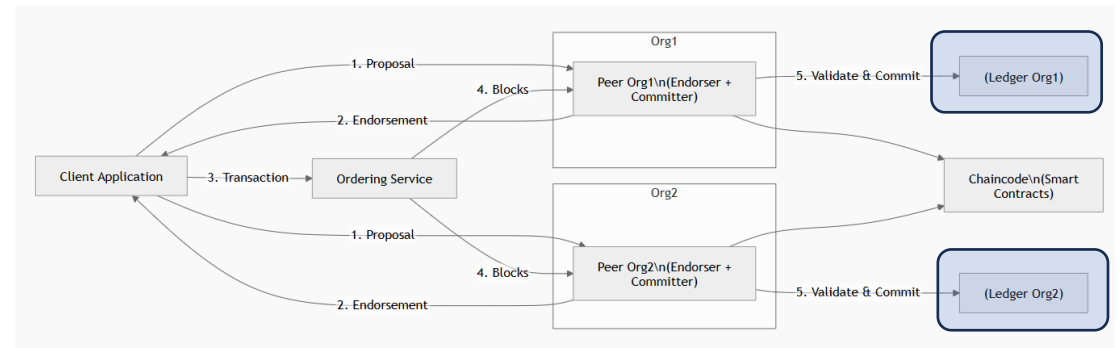
- **Blockchain** → histórico inmutable de todas las transacciones
- **World State** → estado actual de los datos (clave-valor o JSON)

Dónde está

Dentro de cada peer. Cada peer tiene su propia copia.

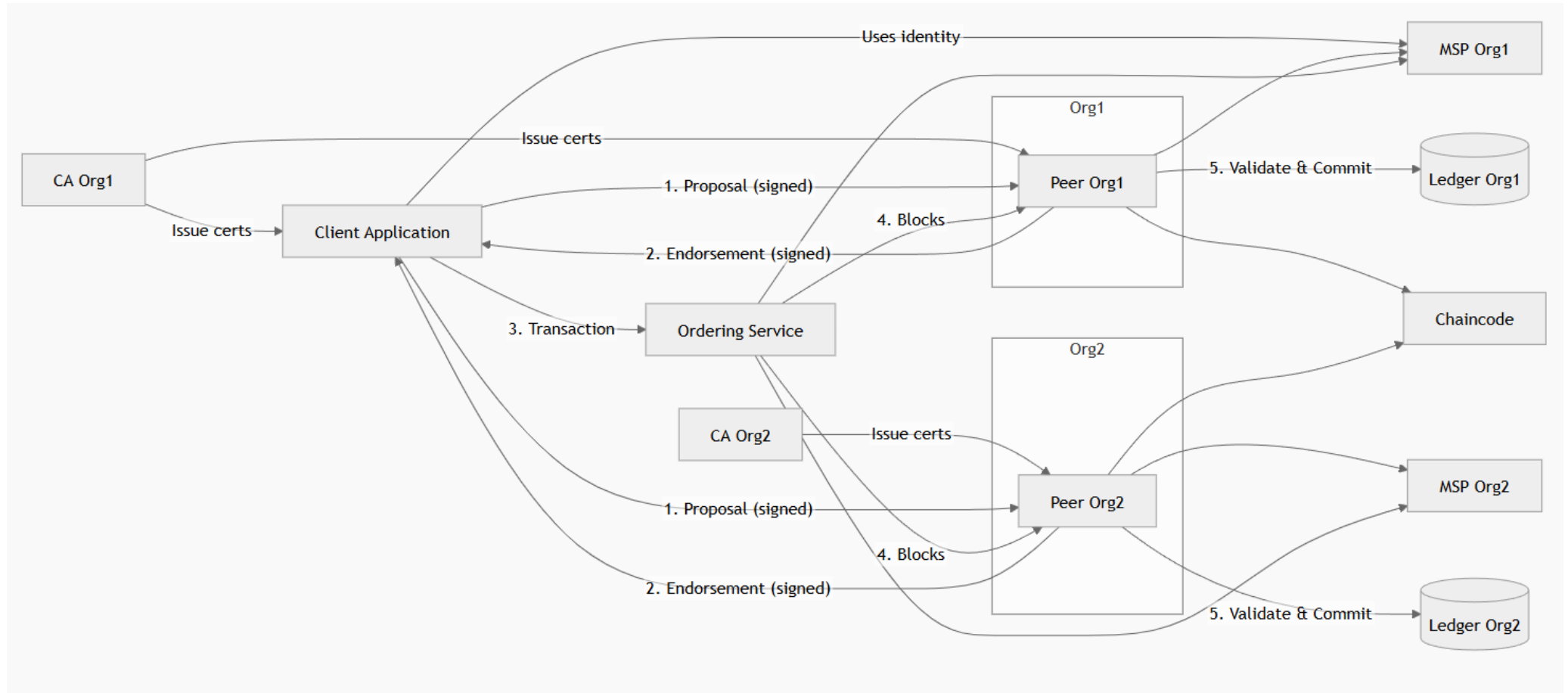
Detalle crucial

Hay un ledger POR CANAL, no un ledger global. Un peer en 2 canales mantiene 2 ledgers distintos.



Idea clave: El ledger es el dato, no el nodo. Lo importante es la información, no el contenedor.

Flujo detallado de una aplicación Fabric



Qué es

El servicio que emite las identidades digitales (certificados X.509) para cada participante.

Qué hace

- Genera certificados X.509
- Identifica usuarios, peers y orderers
- Revoca certificados cuando ya no son válidos

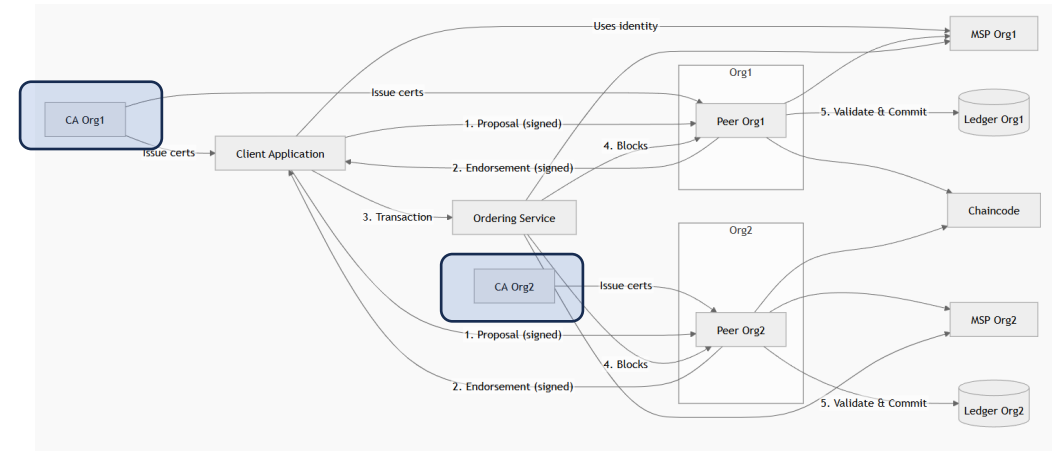
Modelo

Cada organización tiene SU PROPIA CA. No hay una autoridad global que mande sobre todas las organizaciones.

NOTA: EL ordering service suele tener su propia CA aunque, por comodidad, a veces funciona con el de una org existente, aunque centraliza un servicio crítico.

Importante

- X NO valida transacciones
- X NO define la confianza global de la red



Idea clave: La CA emite identidades, pero no decide quién es válido. Eso lo hace el MSP.

MSP — Membership Service Provider

Qué es

El sistema que valida las identidades y define quién está autorizado a operar.

Qué hace

- Define qué CAs son confiables ("acepto los certificados emitidos por la CA de Org1 y de Org2")
- Verifica certificados y firmas en cada operación
- Identifica el rol de cada certificado (peer, client, admin)

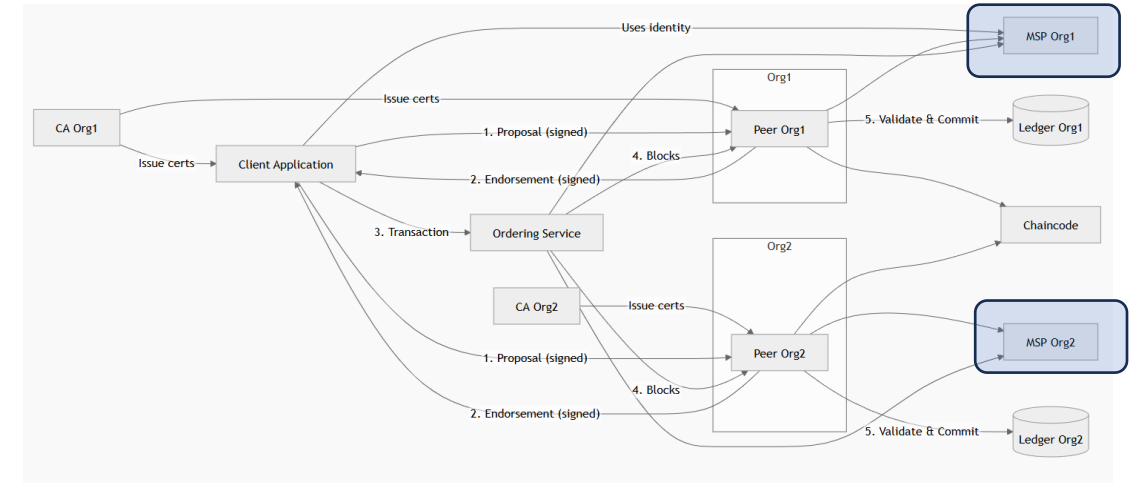
Dónde está

En peers, orderers y clientes. Cada nodo tiene un MSP local.

Detalle clave

El MSP NO es un servicio externo: es configuración. La validación se hace en local sin llamadas a internet.

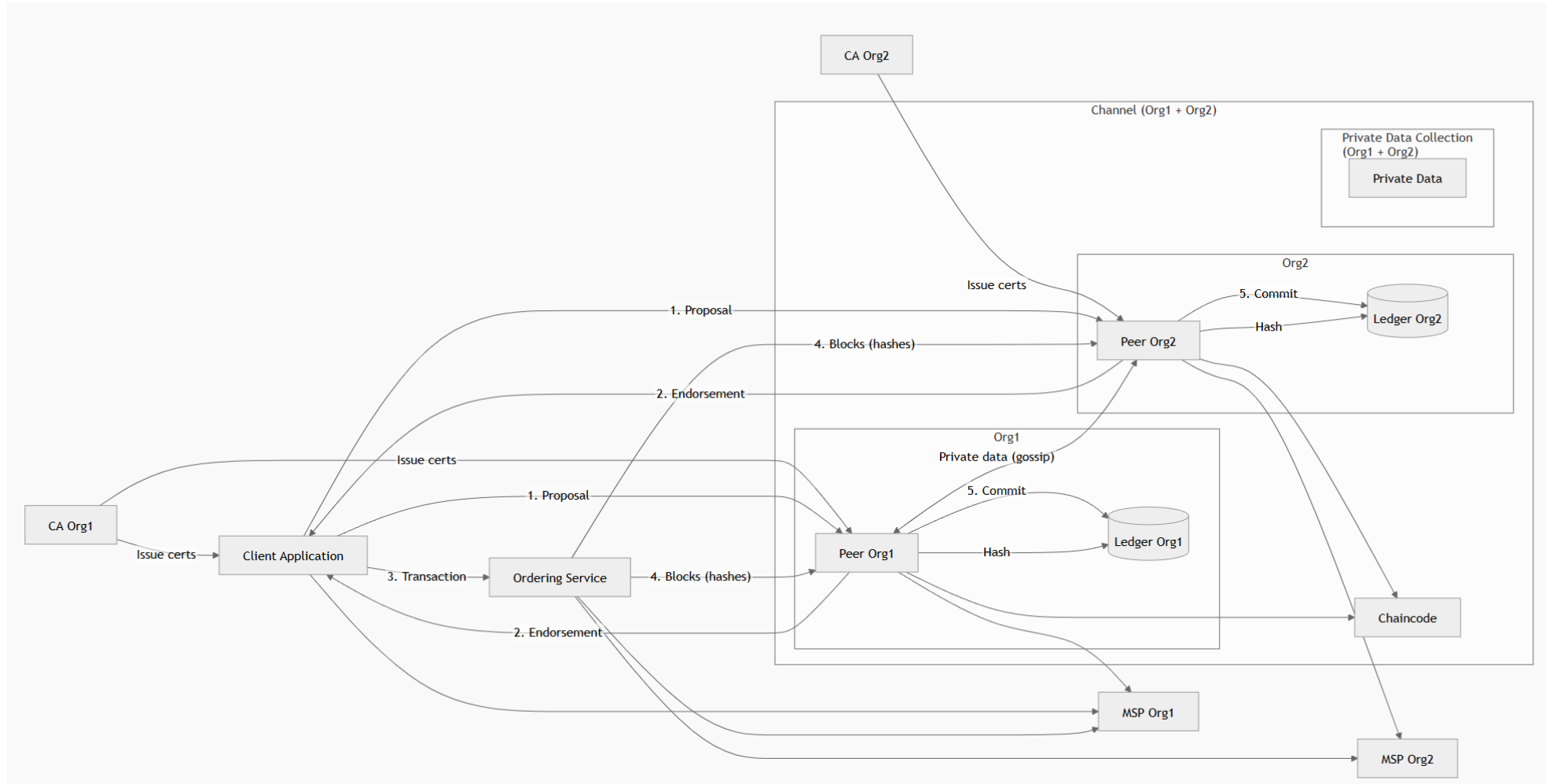
A nivel operativo, un MSP es un conjunto de carpetas y ficheros con certificados, claves y configuración que define qué identidades son válidas dentro de una organización en la red Fabric.



```
msp/  
├─ admincerts/  
├─ cacerts/  
├─ tlscacerts/  
├─ keystore/  
├─ signcerts/  
└─ config.yaml
```

Idea clave: El MSP define la confianza, no la CA. CA emite, MSP decide qué emisor aceptar.

Flujo de una aplicación Fabric con canal



El Canal — una red dentro de la red

Qué es

Una subred privada dentro de la red Fabric, con sus propios participantes y su propio ledger.

Qué define

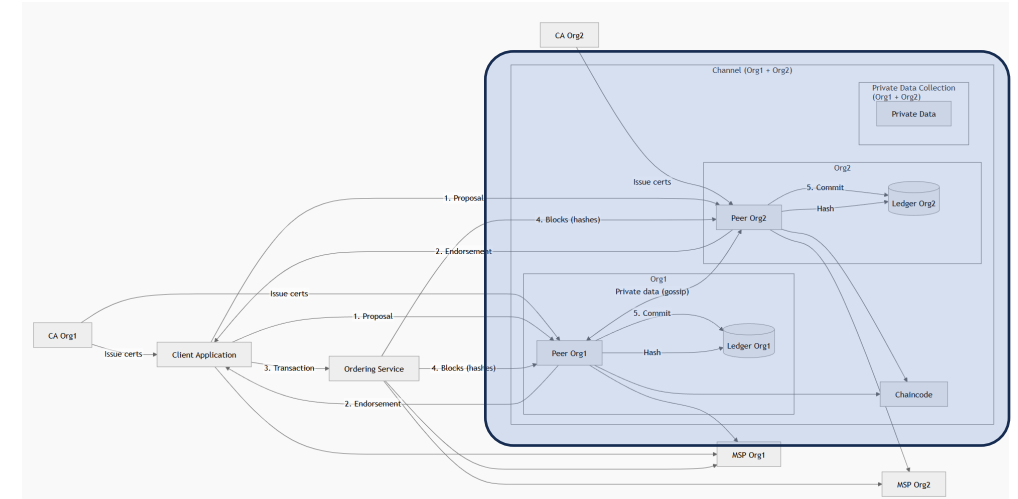
- Quiénes participan (qué organizaciones y qué peers)
- Un ledger independiente del resto
- Qué MSPs son válidos en él
- Su propia configuración de gobernanza

Qué consigue

Aislamiento TOTAL de información. Lo que pasa en un canal no se ve desde otro.

Importante

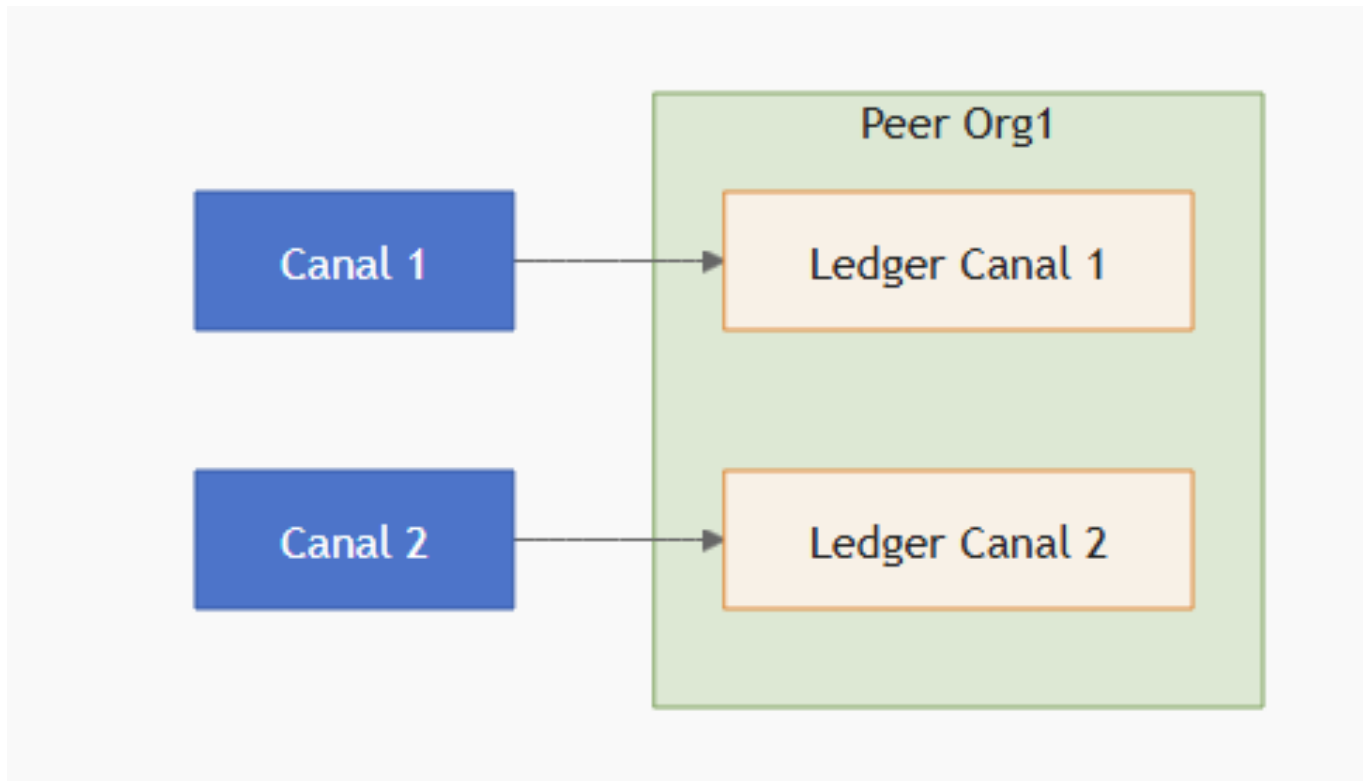
Crear un canal requiere gobernanza (acuerdo entre organizaciones). No es algo que se haga "al vuelo".



Idea clave: Un canal = una red dentro de la red. Es la unidad de privacidad principal de Fabric.

Un canal => Un ledger

- Cada canal tiene su propio ledger
- Cada peer guarda el ledger de los canales en los que participa
- Un peer en 2 canales → 2 ledgers
- No existe un ledger global único



Qué es

Mecanismo para que ciertos datos sean visibles solo para algunos peers DENTRO de un mismo canal.

Qué define

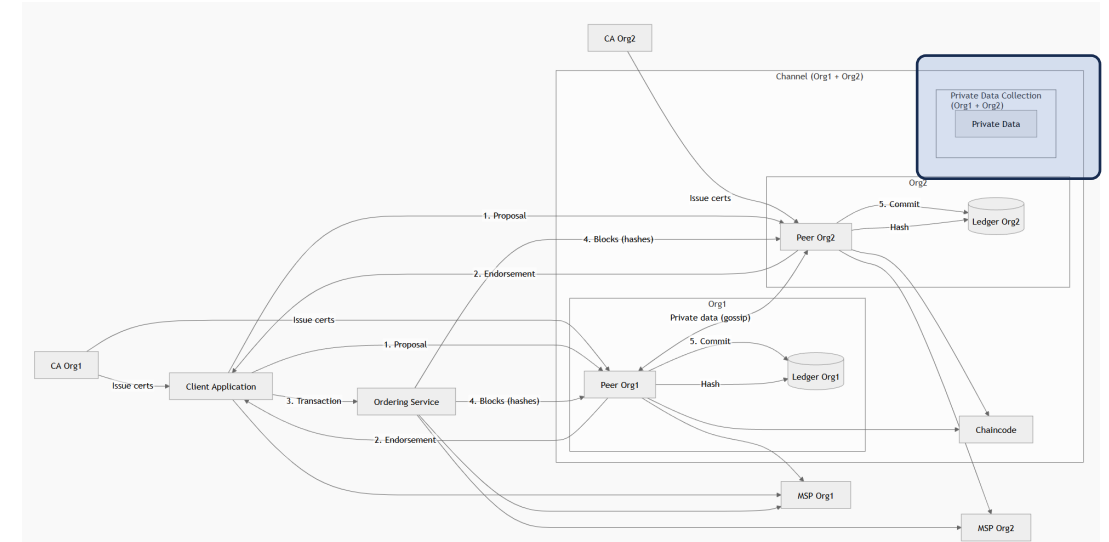
- Qué datos son privados
- Qué organizaciones del canal pueden verlos

Cómo funciona

- Los datos reales solo viajan a los peers autorizados (vía protocolo gossip)
- En el ledger común queda solo un hash, visible para todos
- Esto permite auditoría sin revelar el contenido

Importante

- ✗ NO cifra el dato (controla quién lo recibe, no quién lo lee si lo intercepta)
- ✓ Controla la visibilidad y la distribución



Idea clave: El PDC define QUIÉN ve los datos, no CÓMO viajan.

Canal vs PDC — la pregunta clave



CANAL (Channel)

- Define QUIÉN participa
 - Cada canal tiene:
 - Su propio ledger
 - Su propia configuración
 - Aísla COMPLETAMENTE la información
- Es como crear una red independiente



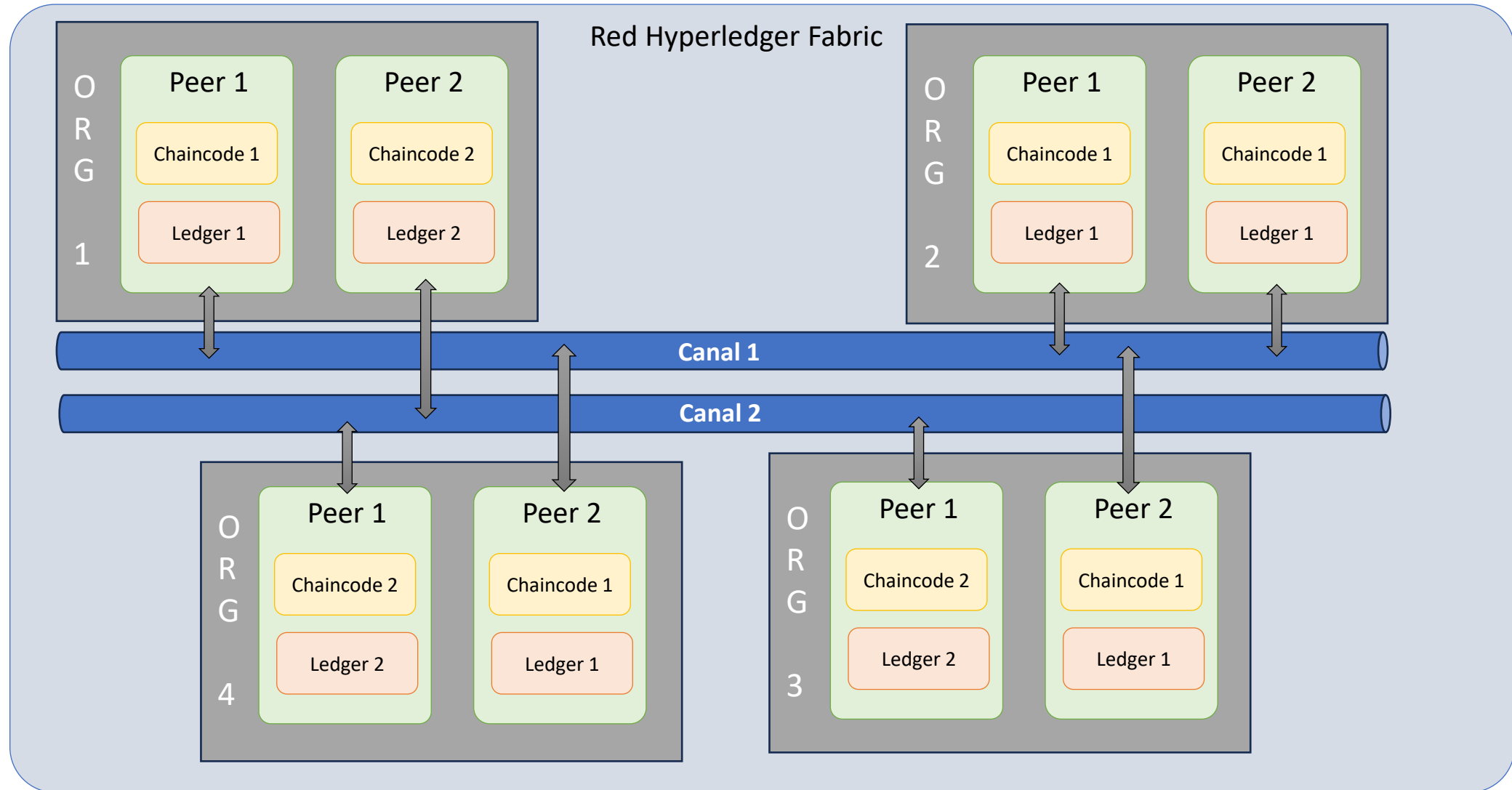
PDC (Private Data Collection)

- Define QUIÉN ve QUÉ datos dentro del canal
 - Solo algunos peers reciben los datos reales
 - El resto solo ve un hash
- Privacidad sin crear nuevos canales

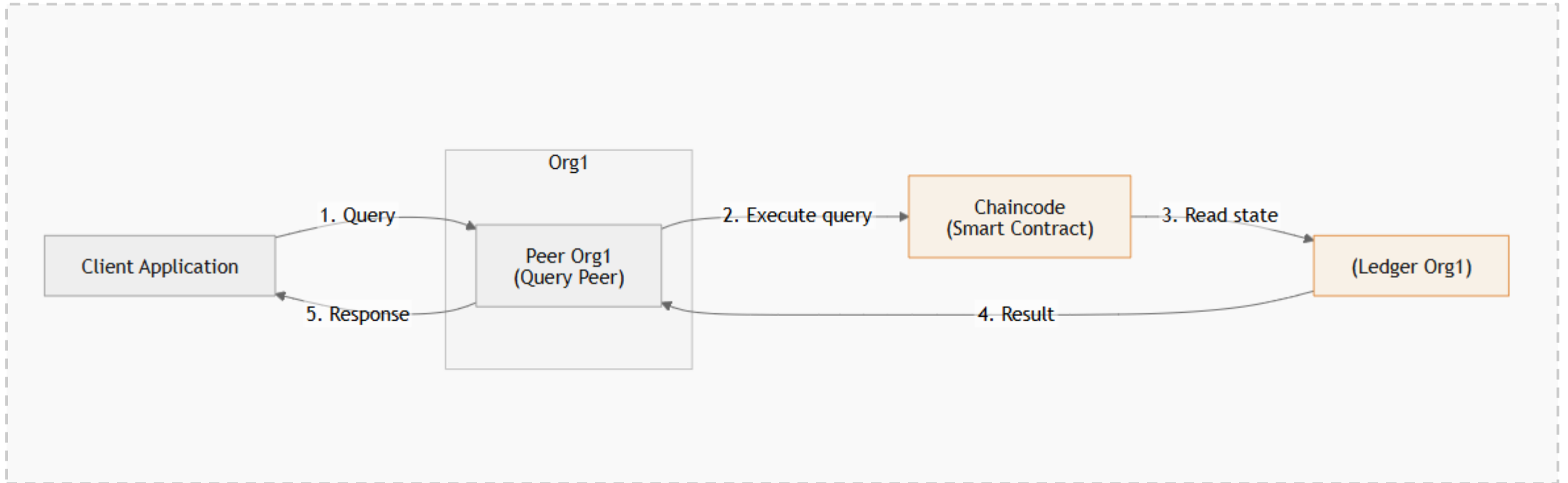


Analogía: El canal es un **grupo de WhatsApp**. El PDC son los **mensajes privados dentro del grupo**. Sin PDC, todos los del grupo ven todo. Con PDC, solo ven lo que les corresponde.

Esquema lógico de varias organizaciones



Una consulta (query) no modifica el estado. No requiere ordenamiento ni distribución a otros peers.

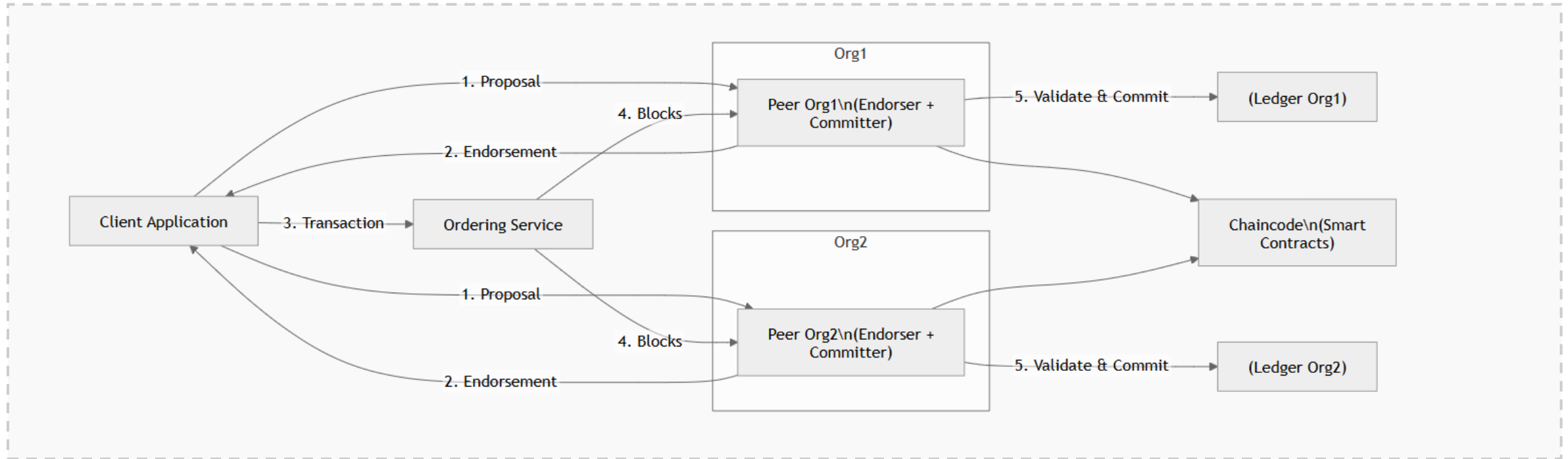


Los 5 pasos

1. El cliente envía una query al peer (1) → 2. El peer ejecuta el chaincode (2) → 3. El chaincode lee el ledger (3) → 4. Devuelve el resultado al peer (4) → 5. El peer responde al cliente (5).

Escritura — Execute, Order, Validate

Una transacción de escritura *SÍ* modifica el estado. Pasa por las 3 fases del modelo Execute-Order-Validate, que es único de Fabric.

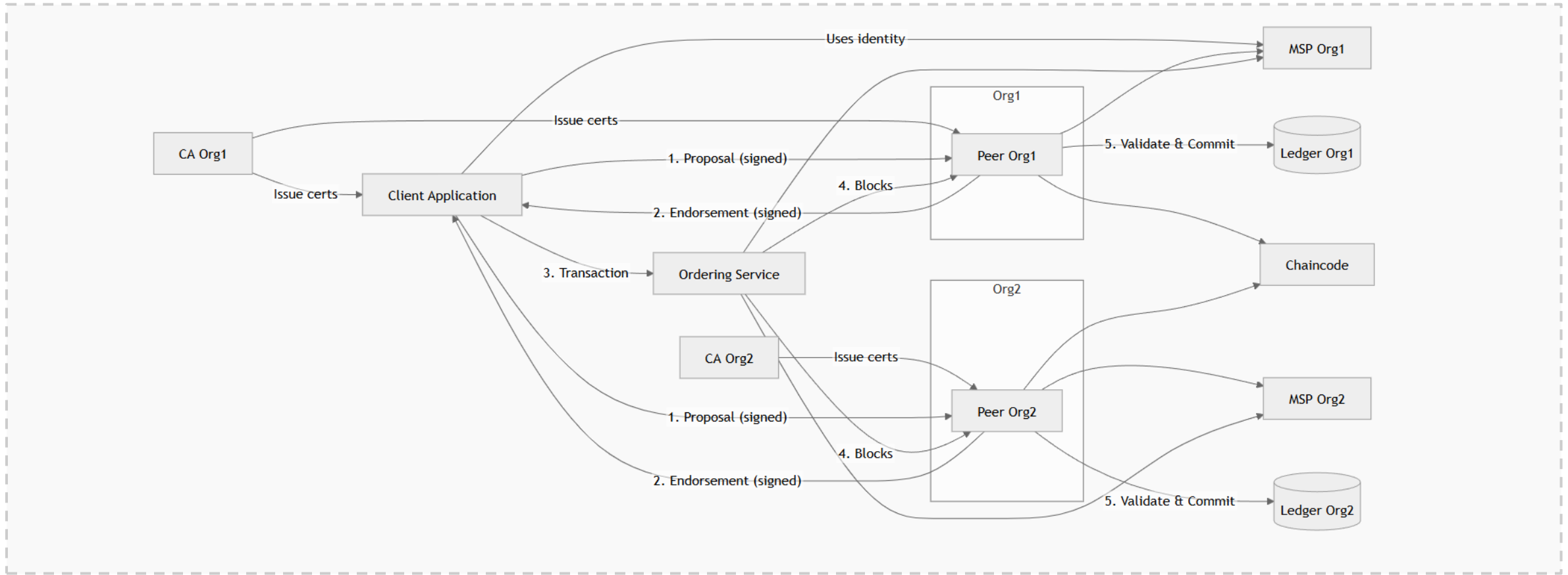


Las 3 fases

- 1. EXECUTE (Endorse):** El cliente envía la propuesta a los endorsers definidos por la **endorsement policy**, que la simulan y la firman. Aquí aún no se modifica nada.
- 2. ORDER:** El cliente envía la transacción endosada al orderer, que la ordena y la empaqueta en un bloque.
- 3. VALIDATE & COMMIT:** Los peers reciben el bloque, validan las firmas y el estado, y finalmente escriben en el ledger.

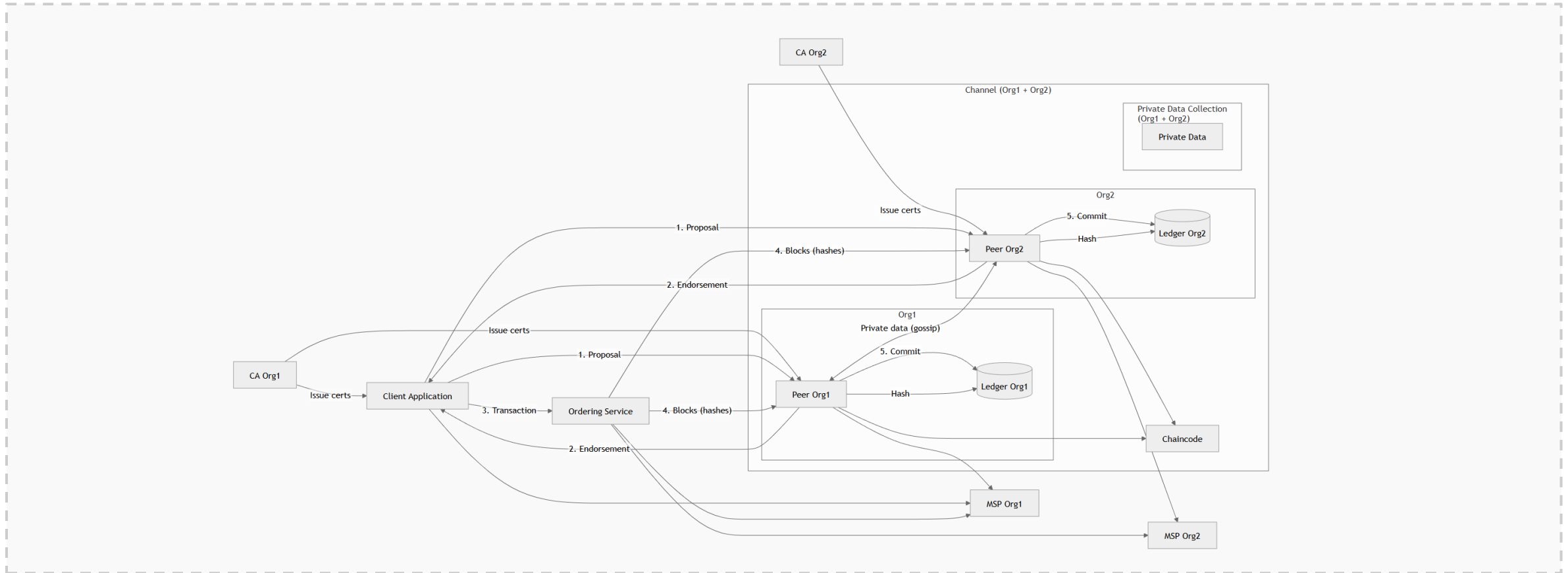
El cuadro completo — con CA, MSP y ledger

Ahora con todas las piezas: identidades emitidas por la CA, validadas por el MSP, y commitadas en el ledger de cada organización.



Si conectas mentalmente todas las flechas, ya conoces cómo funciona Fabric de extremo a extremo.

El mismo flujo, pero ahora dentro de un canal específico y con una Private Data Collection.



Lo que es nuevo respecto al diagrama anterior: el ledger ya no es "el ledger de Org1" sino "el ledger de Org1 en este canal". Y dentro del canal aparece una PDC que solo unas organizaciones pueden ver.

Las 10 ideas con las que te tienes que ir

1. Fabric es blockchain PERMISIONADA. Solo entra quien está autorizado.
2. Cada participante tiene una identidad REAL emitida por una CA con certificados X.509.
3. La CA emite identidades; el MSP decide qué CAs son de confianza.
4. Los PEERS son el motor: ejecutan chaincode, validan transacciones y guardan el ledger.
5. El ORDERER ordena, pero NO decide qué es válido.
6. El CHAINCODE es el smart contract — vive en los peers, escrito en Go, Node.js o Java.
7. La ENDORSEMENT POLICY define qué peers deben firmar para que una transacción valga. NO todos ejecutan, solo los exigidos por la política (gran diferencia con Ethereum).
8. El LEDGER = blockchain inmutable + world state actual. Hay UN ledger por canal.
9. El CANAL es una subred privada con su propio ledger. Cada canal aísla totalmente la información.
10. La PDC permite que ciertos datos solo los vean algunos peers DENTRO del mismo canal.

A. Por que Fabric es diferente

El modelo execute-order-validate en accion.

El problema con order-execute

En Bitcoin y Ethereum las transacciones siguen un orden estricto:

1. El cliente envia la tx.
2. El consenso ordena las tx en bloques.
3. CADA nodo ejecuta cada tx en el orden establecido.

Implicaciones de este modelo:

- Toda la red ejecuta el mismo codigo (sobrecarga global).
- El codigo debe ser determinista (de ahi la EVM y el gas).
- No hay privacidad: todos ven todas las tx.
- Escalabilidad limitada: el throughput es del nodo mas lento.

Order-execute vs Execute-order-validate

Bitcoin / Ethereum (ORDER-EXECUTE)

1. Cliente envia tx

2. Mineros/validadores la incluyen en bloque

3. Consenso ordena los bloques

4. TODOS los nodos ejecutan

5. Estado se actualiza igual en todos

(la red entera ejecuta cada tx)

Hyperledger Fabric (EXECUTE-ORDER-VALIDATE)

1. Cliente envia propuesta a endorsers

2. Endorsers SIMULAN y firman el resultado

3. Cliente envia tx firmada al orderer

4. Orderer ordena tx en bloques

5. Cada peer VALIDA y aplica al world state

(solo endorsers ejecutan; resto valida)

Las tres fases de Fabric en detalle

EXECUTE (simulacion)

El cliente manda la propuesta a un subconjunto de peers (endorsers). Cada endorser ejecuta el chaincode contra su world state, genera read/write set y firma. NO se modifica nada todavia.

ORDER (ordenacion)

El cliente recolecta firmas y manda la tx al ordering service. El orderer NO valida la logica del chaincode: solo establece el orden total y empaqueta las tx en bloques.

VALIDATE & COMMIT

Cada peer recibe el bloque, comprueba la politica de endoso, ejecuta validacion MVCC (read set vs world state actual) y aplica los cambios solo si todo es coherente.

Implicaciones del cambio de modelo

Aspecto	Order-execute (Bitcoin/ETH)	Execute-order-validate (Fabric)
Quien ejecuta	Toda la red	Solo los endorsers indicados
Determinismo	Obligatorio (EVM, gas)	Solo dentro de la simulacion
Lenguaje	Restringido (Solidity, Vyper)	General (Go, Node.js, Java)
Privacidad	Todo es publico	Por canal y por private collection
Coste	Gas en cada nodo	Sin gas; coste operativo
Throughput	Limitado por el nodo mas lento	Escala por endoso paralelo
Quien ordena	Mineros/validadores	Ordering service identificado

Privacidad — Fabric vs Besu (dos filosofías opuestas)

Hyperledger Besu (el cliente Ethereum empresarial) también permite transacciones privadas, pero el modelo es radicalmente distinto al de Fabric. Conviene conocerlo para entender por qué Fabric eligió otro camino:



Fabric — habitaciones privadas

Modelo: creas subredes con su propio libro mayor (canales).

- Cada canal = un ledger COMPLETO independiente
- Los no-miembros del canal NO ven nada (ni el hash)
- Para esconder algo dentro del canal: PDC

La privacidad se diseña con la topología de la red.



Besu — sobres lacrados punto a punto

Modelo: una sola blockchain compartida + transacciones privadas (vía Tessera, antes Orion).

- El payload cifrado viaja directamente entre emisor y receptores
- En la blockchain pública SOLO queda un hash + privateFor
- Cada nodo mantiene un "private state" propio

La privacidad se diseña por transacción.

Idea clave: Fabric crea **habitaciones privadas** donde entrar. Besu envía **sobres lacrados** sobre una sala común. Mismo objetivo, dos filosofías opuestas para conseguirlo.

Cifrar ≠ privacidad: el problema del análisis de metadatos

En Besu el contenido va cifrado, pero la blockchain sigue mostrando mucha información a todos los participantes de la red:

Lo que TODOS ven en la blockchain

- **Hash** del payload cifrado
- **Lista privateFor** → QUIÉN habla con QUIÉN
- **Tamaño** del payload cifrado
- **Timestamp** del bloque
- **Frecuencia** (cuántas tx privadas hay por hora/día)
- **Gas / fee** consumido

Lo que un competidor PUEDE INFERIR

- **Org1 y Org3** tienen relación comercial (privateFor lo dice)
- **Pico de actividad** el día del comunicado de prensa X
- **Tipo de operación** (payload de 4KB ≈ contrato de derivados, 200 bytes ≈ una transferencia simple)
- **Volumen relativo** entre clientes: ¿quién es tu cliente principal?
- **Patrones temporales**: días sin actividad = ¿problemas con ese cliente?

Cómo Fabric resuelve el leak de metadatos

Para esto la PDC NO basta. La solución correcta en Fabric son los CANALES bilaterales.

✗ Con PDC sigues teniendo el problema

La PDC esconde el CONTENIDO de un dato concreto dentro de un canal, pero...

- El **hash** se publica en el ledger del canal
- Todos los miembros del canal ven **CUÁNTAS PDC se crean y CUÁNDO**
- Mismo problema que en Besu: **metadatos visibles**

✓ Canales bilaterales: la solución real

Crear un canal exclusivo entre cada par de participantes que necesite privacidad total:

- **Org1 + Org3** crean un canal solo suyo
- Los demás **NI SIQUIERA SABEN** que ese canal existe
- No ven hashes, ni timestamps, ni volumen — **opacidad total**

El precio a pagar: complejidad operativa. Con N organizaciones que necesitan privacidad bilateral, harían falta hasta $N \times (N-1) / 2$ canales. Cada canal tiene su propia gobernanza, su chaincode desplegado, su ledger. **Es más segura, pero más cara de mantener.**

Por eso en la práctica se combinan: canales compartidos para lo "normal", canales bilaterales para lo "sensible".

Mucha gente piensa que en Fabric 'el orderer ejecuta el chaincode'.

FALSO. El orderer no toca la logica de negocio en ningun momento.

El chaincode se ejecuta UNA SOLA VEZ, durante la simulacion,
y solo en los peers que la politica de endoso exige.

Por eso Fabric puede tener:

- Chaincode en Go arbitrario (no solo EVM).
- Acceso a recursos externos durante la simulacion.
- Un orderer ligero que solo se preocupa del orden.

B. Tipos de peer y politicas

Quien hace que, y quien decide si vale.

Tipos de peer en Fabric

Endorser peer

Peer que tiene chaincode instalado y simula propuestas. Firma el resultado para que cuente como endoso. Cualquier peer con el chaincode puede serlo.

Committer peer

Todos los peers son committers: reciben bloques del orderer, validan endoso y MVCC, y aplican los cambios al world state. Es el rol 'por defecto'.

Anchor peer

Peer publicado en la config del canal como punto de contacto entre organizaciones para iniciar gossip. Típicamente uno o dos por org.

Leader peer

Dentro de una org, el peer que se encarga de pedir bloques al orderer y distribuirlos al resto via gossip. Puede ser estático o elegido dinámicamente.

Un peer NO es 'o endorser o committer'. Suele ser ambos.

Ejemplo tipico de un peer en una org de tamaño medio:

- Es committer (siempre).
- Es endorser (tiene el chaincode instalado).
- Puede ser anchor (declarado en la config del canal).
- Puede ser leader (si gana la eleccion gossip).

El 'tipo' depende del momento y de la configuracion, no del binario.

Que es una policy en Fabric

Una policy es una regla logica sobre identidades que dice 'quien puede X'.

Se aplica en distintos puntos:

- Endorsement policy: quien debe firmar para que una tx valga.
- Channel policies: quien puede unirse, leer, escribir, modificar config.
- ACL: quien puede invocar funciones concretas del chaincode.

Las policies se evaluan contra los certificados X.509 que firma una transaccion y la pertenencia de esos certs a un MSP.

Tipos de policy en Fabric

Tipo	Donde se define	Para que sirve
Signature policy	configtx, chaincode	Combinacion booleana de firmas concretas
Implicit meta policy	configtx	Agregacion de sub-policiess de las orgs (ANY/ALL/MAJORITY)
Endorsement policy	chaincode lifecycle	Firmas necesarias para que una tx sea valida
Channel policy	configtx (canal)	Quien lee, escribe o modifica la config del canal
Lifecycle policy	channel config	Quien aprueba el chaincode y como
ACL	configtx (canal)	Permisos de acceso a recursos especificos del canal

Signature policy: sintaxis

Las roles posibles: admin, member, client, peer, orderer.

```
# Operadores logicos disponibles:
#   AND(a, b)      todos
#   OR(a, b)       cualquiera
#   OutOf(N, ...)  al menos N de la lista

# Solo Org1 (un admin de Org1) firma:
AND('Org1MSP.admin')

# Cualquier miembro de Org1 u Org2:
OR('Org1MSP.member', 'Org2MSP.member')

# 2 de 3 organizaciones (mayoria):
OutOf(2, 'Org1MSP.member',
        'Org2MSP.member',
        'Org3MSP.member')

# Org1 admin Y (Org2 member O Org3 member):
AND('Org1MSP.admin',
    OR('Org2MSP.member', 'Org3MSP.member'))
```

Implicit meta policies

ANY Readers

Basta con que UNA organizacion satisfaga su sub-policy de Readers. Util para canales semi-publicos.

ALL Writers

TODAS las organizaciones deben satisfacer su Writers. Para decisiones que requieren unanimidad.

MAJORITY Admins

Mas de la mitad de las orgs deben firmar como admin. Usado por defecto para cambios de configuracion del canal.

Por que se llaman 'implicitas'

Porque no enumeran a las orgs concretas: se evaluan automaticamente sobre todas las orgs definidas en el canal en ese momento.

Quien aplica cada policy y cuando

Momento	Policy aplicada	Quien la evalua
Endoso (simulacion)	Endorsement policy del cc	Cliente (al recoger firmas)
Validacion del bloque	Endorsement policy del cc	Cada peer al recibir el bloque
Crear / unir canal	Channel/Application/Admins	Orderer y peers
Modificar config canal	Channel/Application/Admins (MAJORITY)	Orderer
Aprobar chaincode	LifecycleEndorsement	Peers de las orgs
Hacer query / invoke	ACL del recurso	Peer destino

Imaginemos la red Fidelity con tres orgs: Hotel, Cafeteria, Auditor.

Endorsement policy del chaincode FidelityPoints:

```
AND('HotelMSP.member', 'CafeteriaMSP.member')
```

-> Para emitir o gastar puntos, hotel y cafeteria firman.

Channel/Application/Admins (modificar config):

MAJORITY Admins -> 2 de 3 admins (Hotel, Cafeteria, Auditor).

Lifecycle endorsement (aprobar nueva version del cc):

MAJORITY Endorsement -> igual que arriba.

Auditor no firma transacciones, pero participa en la gobernanza.